SWIFT 6 INTERVIEW PREPARATION

Software Engineering Interview 7-Day DSA Complete Reference Manual

40 Solved Coding Problems in Swift 6 with Line-by-Line Logic, Xcode Syntax Highlighting & Clickable Problem URLs

---

Target Role: Software Engineer (iOS / Swift)	**Curriculum Length:** 7 Days
Total Solved Programs: 40 Complete Solutions	**Language:** Swift 6
Playground Artifact: 7Day_DSAPrep.playground	**LeetCode Links:** 100% Clickable URLs Included

Curriculum Strategy & Priority Blueprint

This master reference manual contains full Swift 6 implementations and detailed algorithmic logic for all 40 core coding interview problems. Each section provides: (1) **Algorithmic Intuition & Logic**, (2) **Swift Implementation formatted in Xcode Light Theme**, (3) **Formal Time & Space Complexity Analyses**, and (4) **Clickable LeetCode Problem URLs**.

Priority Tier	Core Topics	Target Mastery
Tier 1 (Critical ■■■■■)	Arrays & Hashing, Sliding Window, Trees & BST	15-20 min O(N) execution
Tier 2 (High ■■■■)	Linked Lists, Binary Search, Graphs, Stack	20 min pattern recognition
Tier 3 (Medium ■■■)	Heap / Priority Queue, Dynamic Programming	1D DP & Top K patterns

■ Problem Index & Quick Reference Catalog

#	Problem Title	Curriculum Module	Difficulty
#1	Two Sum	Day 1: Arrays & Hashing	Easy
#2	Contains Duplicate	Day 1: Arrays & Hashing	Easy
#3	Best Time to Buy and Sell Stock	Day 1: Arrays & Hashing	Easy
#4	Valid Anagram	Day 2: Strings, Linked Lists & Stack	Easy
#5	Valid Parentheses	Day 2: Strings, Linked Lists & Stack	Easy
#6	Reverse Linked List	Day 2: Strings, Linked Lists & Stack	Easy
#7	Merge Two Sorted Lists	Day 2: Strings, Linked Lists & Stack	Easy
#8	Binary Search	Day 3: Binary Search, Sorting & Intervals	Easy
#9	Maximum Depth of Binary Tree	Day 4: Trees & Binary Search Trees	Easy
#10	Invert Binary Tree	Day 4: Trees & Binary Search Trees	Easy
#11	Diameter of Binary Tree	Day 4: Trees & Binary Search Trees	Easy
#12	Climbing Stairs	Day 6: Dynamic Programming & Heap	Easy
#13	4. Maximum Subarray (Medium - Kadane's Algorithm)	Day 1: Arrays & Hashing	Medium
#14	Longest Substring Without Repeating Characters	Day 1: Arrays & Hashing	Medium
#15	Product of Array Except Self	Day 1: Arrays & Hashing	Medium
#16	11. Linked List Cycle (Easy - Floyd's Tortoise and Hare)	Day 2: Strings, Linked Lists & Stack	Medium
#17	Remove Nth Node From End of List	Day 2: Strings, Linked Lists & Stack	Medium
#18	Min Stack	Day 2: Strings, Linked Lists & Stack	Medium
#19	Search in Rotated Sorted Array	Day 3: Binary Search, Sorting & Intervals	Medium
#20	Find Minimum in Rotated Sorted Array	Day 3: Binary Search, Sorting & Intervals	Medium
#21	Koko Eating Bananas	Day 3: Binary Search, Sorting & Intervals	Medium
#22	Merge Intervals	Day 3: Binary Search, Sorting & Intervals	Medium
#23	Insert Interval	Day 3: Binary Search, Sorting & Intervals	Medium
#24	Non-Overlapping Intervals	Day 3: Binary Search, Sorting & Intervals	Medium
#25	Binary Tree Level Order Traversal	Day 4: Trees & Binary Search Trees	Medium
#26	Validate Binary Search Tree	Day 4: Trees & Binary Search Trees	Medium
#27	Lowest Common Ancestor	Day 4: Trees & Binary Search Trees	Medium
#28	Kth Smallest Element in a BST	Day 4: Trees & Binary Search Trees	Medium
#29	Number of Islands	Day 5: Graphs	Medium
#30	Clone Graph	Day 5: Graphs	Medium
#31	30. Course Schedule (Medium - Topological Sort)	Day 5: Graphs	Medium
#32	31. Rotting Oranges (Medium - Multi-Source BFS)	Day 5: Graphs	Medium
#33	Pacific Atlantic Water Flow	Day 5: Graphs	Medium
#34	Graph Valid Tree	Day 5: Graphs	Medium
#35	House Robber	Day 6: Dynamic Programming & Heap	Medium

#	Problem Title	Curriculum Module	Difficulty
#36	Coin Change	Day 6: Dynamic Programming & Heap	Medium
#37	37. Longest Increasing Subsequence (Medium - Patience Sorting)	Day 6: Dynamic Programming & Heap	Medium
#38	Word Break	Day 6: Dynamic Programming & Heap	Medium
#39	39. Kth Largest Element in an Array (Medium - QuickSelect)	Day 6: Dynamic Programming & Heap	Medium
#40	40. Top K Frequent Elements (Medium - Bucket Sort)	Day 6: Dynamic Programming & Heap	Medium

■ EASY LEVEL PROBLEMS

• Day 1: Arrays & Hashing

#1. Two Sum (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Use a Hash Map mapping element values to their indices. For each number, compute complement = target - num. If complement exists in map, return indices immediately. Otherwise, store current num in map.*

```swift
class TwoSum {
    static func solve(_ nums: [Int], _ target: Int) -> [Int] {
        var map = [Int: Int]()
        for (i, num) in nums.enumerated() {
            let complement = target - num
            if let complementIndex = map[complement] {
                return [complementIndex, i]
            }
            map[num] = i
        }
        return []
    }
}
```

Time Complexity: O(N) - Single pass over array of length N

Space Complexity: O(N) - Hash map storing up to N entries

#2. Contains Duplicate (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Use a Hash Set to track visited elements. As we iterate through the array, check if current element is already present in the set. If yes, duplicate exists. Otherwise, insert element into set.*

```swift
class ContainsDuplicate {
    static func solve(_ nums: [Int]) -> Bool {
        var seen = Set<Int>()
        for num in nums {
            if seen.contains(num) { return true }
            seen.insert(num)
        }
        return false
    }
}
```

Time Complexity: O(N) - Single pass over N elements

Space Complexity: O(N) - Space for Set storing unique values

#3. Best Time to Buy and Sell Stock (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Maintain a running minimum purchase price minPrice. Iterate through stock prices: compute potential profit price - minPrice and update global maxProfit. Update minPrice whenever a lower price is found.*

```swift
class BuySellStock {
    static func solve(_ prices: [Int]) -> Int {
        var minPrice = Int.max, maxProfit = 0
        for price in prices {
            minPrice = min(minPrice, price)
            maxProfit = max(maxProfit, price - minPrice)
        }
        return maxProfit
    }
}
```

Time Complexity: O(N) - Single linear pass **Space Complexity:** O(1) - Constant auxiliary space

• Day 2: Strings, Linked Lists & Stack

#4. Valid Anagram (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Count character occurrences in string s using a hash map or frequency array. Decrement counts while iterating through string t. Return false if count drops below 0 or lengths differ.*

```swift
class ValidAnagram {
    static func solve(_ s: String, _ t: String) -> Bool {
        guard s.count == t.count else { return false }
        var counts = [Character: Int]()
        for char in s { counts[char, default: 0] += 1 }
        for char in t {
            guard let count = counts[char], count > 0 else { return false }
            counts[char] = count - 1
        }
        return true
    }
}
```

Time Complexity: O(N) - Linear pass over both strings **Space Complexity:** O(1) - Fixed 26-letter frequency table

#5. Valid Parentheses (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Push opening brackets onto stack. For closing brackets, check if stack is non-empty and matching opening bracket is at stack top. If valid, pop top; otherwise return false. Check stack is empty at end.*

```swift
class ValidParentheses {
    static func solve(_ s: String) -> Bool {
        var stack = [Character]()
        let map: [Character: Character] = [")": "(", "]": "[", "}": "{"]
        for char in s {
            if let expected = map[char] {
                if stack.isEmpty || stack.removeLast() != expected { return false }
            } else {
                stack.append(char)
            }
        }
        return stack.isEmpty
    }
}
```

Time Complexity: O(N) - Single pass over string length N **Space Complexity:** O(N) - Stack storing up to N brackets

#6. Reverse Linked List (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Iterative 3-pointer strategy: maintain prev (starts nil) and curr (starts head). Save nextTemp = curr.next, set curr.next = prev, then advance prev = curr and curr = nextTemp. Return prev.*

```swift
class ReverseLinkedList {
    static func solve(_ head: ListNode?) -> ListNode? {
        var prev: ListNode? = nil, curr = head
        while curr != nil {
            let nextTemp = curr?.next
            curr?.next = prev
            prev = curr
            curr = nextTemp
        }
        return prev
    }
}
```

Time Complexity: O(N) - Single pass over linked list nodes **Space Complexity:** O(1) - Constant auxiliary pointer space

#7. Merge Two Sorted Lists (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Use dummy head node with pointer tail. Compare values p1.val vs p2.val, attach smaller node to tail.next, and advance that list pointer. Attach remaining non-nil list at end.*

```swift
class MergeTwoLists {
    static func solve(_ l1: ListNode?, _ l2: ListNode?) -> ListNode? {
        let dummy = ListNode(0)
        var tail = dummy, p1 = l1, p2 = l2
        while let n1 = p1, let n2 = p2 {
            if n1.val <= n2.val { tail.next = n1; p1 = n1.next }
            else { tail.next = n2; p2 = n2.next }
            tail = tail.next!
        }
        tail.next = p1 ?? p2
        return dummy.next
    }
}
```

Time Complexity: O(N + M) - Proportional to nodes in both lists

Space Complexity: O(1) - Constant extra pointers

• Day 3: Binary Search, Sorting & Intervals

#8. Binary Search (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Maintain left and right pointers. Calculate mid = left + (right - left) / 2 to avoid integer overflow. Compare nums[mid] with target to halve search range at each step.*

```swift
class BinarySearch {
    static func solve(_ nums: [Int], _ target: Int) -> Int {
        var left = 0, right = nums.count - 1
        while left <= right {
            let mid = left + (right - left) / 2
            if nums[mid] == target { return mid }
            else if nums[mid] < target { left = mid + 1 }
            else { right = mid - 1 }
        }
        return -1
    }
}
```

Time Complexity: O(log N) - Halves search space per step

Space Complexity: O(1) - Iterative search

• Day 4: Trees & Binary Search Trees

#9. Maximum Depth of Binary Tree (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Recursion base case: nil node has depth 0. For non-nil node, depth is 1 + max(depth(left), depth(right)). Postorder DFS traversal.*

```
class MaxDepthBinaryTree {
    static func maxDepth(_ root: TreeNode?) -> Int {
        guard let root = root else { return 0 }
        return 1 + max(maxDepth(root.left), maxDepth(root.right))
    }
}
```

Time Complexity: O(N) - Visits each tree node once

Space Complexity: O(H) - Recursion call stack proportional to height H

#10. Invert Binary Tree (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Recursively swap left and right children: save root.left, set root.left = invertTree(root.right) and root.right = invertTree(temp). Return root.*

```
class InvertBinaryTree {
    static func invertTree(_ root: TreeNode?) -> TreeNode? {
        guard let root = root else { return nil }
        let temp = root.left
        root.left = invertTree(root.right)
        root.right = invertTree(temp)
        return root
    }
}
```

Time Complexity: O(N) - Visits all nodes

Space Complexity: O(H) - Call stack space

#11. Diameter of Binary Tree (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *At each node, the longest path passing through it is leftDepth + rightDepth. Maintain a global maxDiameter. Recursive depth helper returns 1 + max(leftDepth, rightDepth).*

```
class DiameterBinaryTree {
    static func diameterOfBinaryTree(_ root: TreeNode?) -> Int {
        var maxDia = 0
        func depth(_ node: TreeNode?) -> Int {
            guard let node = node else { return 0 }
            let l = depth(node.left), r = depth(node.right)
            maxDia = max(maxDia, l + r)
            return 1 + max(l, r)
        }
        _ = depth(root)
        return maxDia
    }
}
```

Time Complexity: O(N) - Single pass over tree

Space Complexity: O(H) - Recursion depth H

• Day 6: Dynamic Programming & Heap

#12. Climbing Stairs (Easy) [■ Open LeetCode Problem]

Logic & Intuition: *Fibonacci DP transition: dp[i] = dp[i-1] + dp[i-2]. Space optimized to two rolling variables first and second representing previous step combinations.*

```swift
class ClimbingStairs {
    static func climbStairs(_ n: Int) -> Int {
        if n <= 2 { return n }
        var first = 1, second = 2
        for _ in 3...n {
            let third = first + second
            first = second; second = third
        }
        return second
    }
}
```

Time Complexity: O(N) - Linear iteration up to stair N **Space Complexity:** O(1) - Constant variables

■ MEDIUM LEVEL PROBLEMS

• Day 1: Arrays & Hashing

#13. 4. Maximum Subarray (Medium - Kadane's Algorithm) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Kadane's algorithm: At each element, decide whether to append element to existing running subarray sum (currentSum + num) or start a fresh subarray from num. Track global maximum maxSoFar across all steps.*

```swift
class MaxSubarray {
    static func solve(_ nums: [Int]) -> Int {
        guard !nums.isEmpty else { return 0 }
        var maxSoFar = nums[0], currentSum = nums[0]
        for num in nums.dropFirst() {
            currentSum = max(num, currentSum + num)
            maxSoFar = max(maxSoFar, currentSum)
        }
        return maxSoFar
    }
}
```

Time Complexity: O(N) - Linear pass through array **Space Complexity:** O(1) - Constant memory variables

#14. Longest Substring Without Repeating Characters (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Sliding window with two pointers left and right. Maintain a hash map of character -> last seen index. When a duplicate character is encountered inside window (lastSeen >= left), jump left past lastSeen.*

```swift
class LongestSubstringWithoutRepeating {
    static func solve(_ s: String) -> Int {
        let chars = Array(s)
        var map = [Character: Int](), left = 0, maxLen = 0
        for (right, char) in chars.enumerated() {
            if let lastPos = map[char], lastPos >= left {
                left = lastPos + 1
            }
            map[char] = right
            maxLen = max(maxLen, right - left + 1)
        }
        return maxLen
    }
}
```

Time Complexity: O(N) - Linear sweep of string length N

Space Complexity: O(min(N, M)) - Space for character map

#15. Product of Array Except Self (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Two prefix passes without division. Pass 1: compute prefix products from left to right into result array. Pass 2: compute suffix products from right to left while multiplying directly into result array.*

```swift
class ProductExceptSelf {
    static func solve(_ nums: [Int]) -> [Int] {
        let n = nums.count
        var result = Array(repeating: 1, count: n)
        var leftProd = 1
        for i in 0..<n {
            result[i] = leftProd; leftProd *= nums[i]
        }
        var rightProd = 1
        for i in (0..<n).reversed() {
            result[i] *= rightProd; rightProd *= nums[i]
        }
        return result
    }
}
```

Time Complexity: O(N) - Two linear passes

Space Complexity: O(1) - Output array does not count as extra space

• Day 2: Strings, Linked Lists & Stack

#16. 11. Linked List Cycle (Easy - Floyd's Tortoise and Hare) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Fast & Slow pointer pattern. Advance slow pointer by 1 step and fast pointer by 2 steps. If a cycle exists, fast will eventually meet slow (slow === fast). If fast hits nil, no cycle exists.*

```
class LinkedListCycle {
    static func hasCycle(_ head: ListNode?) -> Bool {
        var slow = head, fast = head
        while fast != nil && fast?.next != nil {
            slow = slow?.next
            fast = fast?.next?.next
            if slow === fast { return true }
        }
        return false
    }
}
```

Time Complexity: O(N) - Fast pointer travels at most 2N steps

Space Complexity: O(1) - Two extra pointers

#17. Remove Nth Node From End of List (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Two pointers first and second initialized at dummy node. Advance first by n + 1 steps to create a gap of n nodes between them. Move both pointers together until first reaches nil. Unlink node.*

```
class RemoveNthFromEnd {
    static func solve(_ head: ListNode?, _ n: Int) -> ListNode? {
        let dummy = ListNode(0, head)
        var first: ListNode? = dummy, second: ListNode? = dummy
        for _ in 0...n { first = first?.next }
        while first != nil {
            first = first?.next
            second = second?.next
        }
        second?.next = second?.next?.next
        return dummy.next
    }
}
```

Time Complexity: O(N) - Single pass over list

Space Complexity: O(1) - Constant pointer space

#18. Min Stack (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Maintain two parallel stacks: mainStack for storing values, and minStack for storing the minimum value present up to that depth. On push, push min(val, minStack.last) onto minStack.*

```swift
class MinStack {
    private var mainStack = [Int](), minStack = [Int]()
    func push(_ val: Int) {
        mainStack.append(val)
        let curMin = minStack.last ?? Int.max
        minStack.append(min(val, curMin))
    }
    func pop() { _ = mainStack.popLast(); _ = minStack.popLast() }
    func top() -> Int { return mainStack.last ?? -1 }
    func getMin() -> Int { return minStack.last ?? -1 }
}
```

Time Complexity: O(1) - Constant time for push, pop, top, getMin

Space Complexity: O(N) - Dual stacks storing N values

• Day 3: Binary Search, Sorting & Intervals

#19. Search in Rotated Sorted Array (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Binary search on rotated array: At mid, check if left half nums[left...mid] is sorted (nums[left] <= nums[mid]). If yes, check if target lies in left range; else search right half.*

```swift
class SearchRotated {
    static func solve(_ nums: [Int], _ target: Int) -> Int {
        var left = 0, right = nums.count - 1
        while left <= right {
            let mid = left + (right - left) / 2
            if nums[mid] == target { return mid }
            if nums[left] <= nums[mid] {
                if nums[left] <= target && target < nums[mid] { right = mid - 1 }
                else { left = mid + 1 }
            } else {
                if nums[mid] < target && target <= nums[right] { left = mid + 1 }
                else { right = mid - 1 }
            }
        }
        return -1
    }
}
```

Time Complexity: O(log N) - Modified binary search

Space Complexity: O(1) - Constant variables

#20. Find Minimum in Rotated Sorted Array (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Compare nums[mid] with right boundary nums[right]. If nums[mid] > nums[right], minimum lies strictly in right half (left = mid + 1). Otherwise minimum is at mid or in left half (right = mid).*

```swift
class FindMinRotated {
    static func solve(_ nums: [Int]) -> Int {
        var left = 0, right = nums.count - 1
        while left < right {
            let mid = left + (right - left) / 2
            if nums[mid] > nums[right] { left = mid + 1 }
            else { right = mid }
        }
        return nums[left]
    }
}
```

Time Complexity: O(log N) - Logarithmic search

Space Complexity: O(1) - Constant extra space

#21. Koko Eating Bananas (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Binary search on eating speed k in range 1...maxPile. For candidate speed k, compute total hours spent using ceiling division (pile + k - 1) / k. If hours <= h, try smaller speed.*

```swift
class KokoEatingBananas {
    static func minEatingSpeed(_ piles: [Int], _ h: Int) -> Int {
        var left = 1, right = piles.max() ?? 1, result = right
        while left <= right {
            let k = left + (right - left) / 2
            var hours = 0
            for pile in piles { hours += (pile + k - 1) / k }
            if hours <= h { result = k; right = k - 1 }
            else { left = k + 1 }
        }
        return result
    }
}
```

Time Complexity: O(N log(MaxPile)) - N piles * binary search on max pile

Space Complexity: O(1) - Constant auxiliary space

#22. Merge Intervals (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Sort intervals by start times start_i. Maintain currentInterval. Iterate sorted list: if next.start <= current.end, merge by setting current.end = max(current.end, next.end). Else push current and update.*

```
class MergeIntervals {
    static func solve(_ intervals: [[Int]]) -> [[Int]] {
        guard intervals.count > 1 else { return intervals }
        let sorted = intervals.sorted { $0[0] < $1[0] }
        var res = [[Int]](), curr = sorted[0]
        for next in sorted.dropFirst() {
            if next[0] <= curr[1] { curr[1] = max(curr[1], next[1]) }
            else { res.append(curr); curr = next }
        }
        res.append(curr)
        return res
    }
}
```

Time Complexity: O(N log N) - Sorting intervals takes N log N

Space Complexity: O(N) - Storage for merged result

#23. Insert Interval (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *3 Phase processing: (1) Add all intervals ending before newInterval.start. (2) Merge all overlapping intervals with newInterval by expanding bounds. (3) Add all remaining intervals starting after.*

```
class InsertInterval {
    static func solve(_ intervals: [[Int]], _ newInterval: [Int]) -> [[Int]] {
        var res = [[Int]](), newInt = newInterval, i = 0, n = intervals.count
        while i < n && intervals[i][1] < newInt[0] { res.append(intervals[i]); i += 1 }
        while i < n && intervals[i][0] <= newInt[1] {
            newInt[0] = min(newInt[0], intervals[i][0])
            newInt[1] = max(newInt[1], intervals[i][1])
            i += 1
        }
        res.append(newInt)
        while i < n { res.append(intervals[i]); i += 1 }
        return res
    }
}
```

Time Complexity: O(N) - Single linear pass over sorted intervals

Space Complexity: O(N) - Output array space

#24. Non-Overlapping Intervals (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Greedy choice: Sort intervals by end times end_i. Maintain lastEnd. When an overlapping interval is found (interval.start < lastEnd), increment removal count. Otherwise update lastEnd = interval.end.*

```
class NonOverlappingIntervals {
    static func eraseOverlapIntervals(_ intervals: [[Int]]) -> Int {
        guard !intervals.isEmpty else { return 0 }
        let sorted = intervals.sorted { $0[1] < $1[1] }
        var removals = 0, lastEnd = sorted[0][1]
        for interval in sorted.dropFirst() {
            if interval[0] < lastEnd { removals += 1 }
            else { lastEnd = interval[1] }
        }
        return removals
    }
}
```

Time Complexity: O(N log N) - Sorting intervals

Space Complexity: O(1) - Constant variables

• Day 4: Trees & Binary Search Trees

#25. Binary Tree Level Order Traversal (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *BFS Queue traversal. Process level by level: snapshot current queue.count, dequeue nodes of current level into level array, and enqueue non-nil left and right children.*

```
class LevelOrderTraversal {
    static func solve(_ root: TreeNode?) -> [[Int]] {
        guard let root = root else { return [] }
        var res = [[Int]](), queue = [root]
        while !queue.isEmpty {
            let levelSize = queue.count
            var level = [Int]()
            for _ in 0..<levelSize {
                let node = queue.removeFirst()
                level.append(node.val)
                if let l = node.left { queue.append(l) }
                if let r = node.right { queue.append(r) }
            }
            res.append(level)
        }
        return res
    }
}
```

Time Complexity: O(N) - Dequeues every node once

Space Complexity: O(N) - Queue holds max nodes at tree level

#26. Validate Binary Search Tree (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Recursive bound validation: pass minVal and maxVal down recursive calls. Left child must satisfy val < parent.val, right child must satisfy val > parent.val.*

```swift
class ValidateBST {
    static func isValidBST(_ root: TreeNode?) -> Bool { return helper(root, nil, nil) }
    private static func helper(_ node: TreeNode?, _ min: Int?, _ max: Int?) -> Bool {
        guard let node = node else { return true }
        if let min = min, node.val <= min { return false }
        if let max = max, node.val >= max { return false }
        return helper(node.left, min, node.val) && helper(node.right, node.val, max)
    }
}
```

Time Complexity: O(N) - Visits each node once

Space Complexity: O(H) - Call stack depth H

#27. Lowest Common Ancestor (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Recursive bottom-up search: if root is nil, p, or q, return root. Recurse left and right subtrees. If both left and right return non-nil, root is the LCA. Otherwise return non-nil branch.*

```swift
class LowestCommonAncestor {
    static func solve(_ root: TreeNode?, _ p: TreeNode?, _ q: TreeNode?) -> TreeNode? {
        if root == nil || root === p || root === q { return root }
        let left = solve(root?.left, p, q), right = solve(root?.right, p, q)
        if left != nil && right != nil { return root }
        return left != nil ? left : right
    }
}
```

Time Complexity: O(N) - Worst case visits all nodes

Space Complexity: O(H) - Call stack depth

#28. Kth Smallest Element in a BST (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Iterative Inorder traversal (Left -> Node -> Right) using a stack. Nodes are popped in strictly ascending order. Increment counter on pop; when counter == k, return node value.*

```swift
class KthSmallestBST {
    static func kthSmallest(_ root: TreeNode?, _ k: Int) -> Int {
        var stack = [TreeNode](), curr = root, count = 0
        while curr != nil || !stack.isEmpty {
            while let node = curr { stack.append(node); curr = node.left }
            let node = stack.removeLast()
            count += 1
            if count == k { return node.val }
            curr = node.right
        }
        return -1
    }
}
```

Time Complexity: O(H + K) - Traverses to left leaf and pops K nodes

Space Complexity: O(H) - Stack depth H

• Day 5: Graphs

#29. Number of Islands (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Iterate 2D grid cell by cell. When a land cell '1' is found, increment island counter and launch DFS to sink all connected land cells by mutating them to '0'.*

```swift
class NumberOfIslands {
    static func numIslands(_ grid: [[Character]]) -> Int {
        guard !grid.isEmpty else { return 0 }
        var g = grid, rows = g.count, cols = g[0].count, count = 0
        for r in 0..<rows {
            for c in 0..<cols {
                if g[r][c] == "1" { count += 1; dfs(&g, r, c, rows, cols) }
            }
        }
        return count
    }
    private static func dfs(_ g: inout [[Character]], _ r: Int, _ c: Int, _ R: Int, _ C: Int) {
        if r < 0 || c < 0 || r >= R || c >= C || g[r][c] == "0" { return }
        g[r][c] = "0"
        dfs(&g, r+1, c, R, C); dfs(&g, r-1, c, R, C)
        dfs(&g, r, c+1, R, C); dfs(&g, r, c-1, R, C)
    }
}
```

Time Complexity: O(M * N) - Every grid cell visited at most a constant number of times

Space Complexity: O(M * N) - Worst case recursion depth for all land cells

#30. Clone Graph (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Deep copy undirected graph using DFS and a visited dictionary mapping original node value -> cloned Node. For each neighbor of current node, attach recursively cloned neighbor.*

```swift
class CloneGraph {
    static func clone(_ node: Node?) -> Node? {
        guard let node = node else { return nil }
        var visited = [Int: Node]()
        func dfs(_ curr: Node) -> Node {
            if let cloned = visited[curr.val] { return cloned }
            let copy = Node(curr.val)
            visited[curr.val] = copy
            for n in curr.neighbors { copy.neighbors.append(dfs(n)) }
            return copy
        }
        return dfs(node)
    }
}
```

Time Complexity: O(V + E) - Visits all vertices V and edges E

Space Complexity: O(V) - Visited dictionary storing V nodes

#31. 30. Course Schedule (Medium - Topological Sort) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Kahn's Algorithm BFS: Build adjacency list and in-degree count for each course. Enqueue all courses with in-degree 0. Dequeue course, increment processed count, and decrement in-degree of neighbors.*

```swift
class CourseSchedule {
    static func canFinish(_ numCourses: Int, _ pre: [[Int]]) -> Bool {
        var inDeg = Array(repeating: 0, count: numCourses), adj = [Int: [Int]]()
        for p in pre { adj[p[1], default: []].append(p[0]); inDeg[p[0]] += 1 }
        var q = (0..<numCourses).filter { inDeg[$0] == 0 }, count = 0
        while !q.isEmpty {
            let curr = q.removeFirst(); count += 1
            for n in adj[curr] ?? [] {
                inDeg[n] -= 1
                if inDeg[n] == 0 { q.append(n) }
            }
        }
        return count == numCourses
    }
}
```

Time Complexity: O(V + E) - Processing nodes and edges

Space Complexity: O(V + E) - Adjacency list and in-degree table

#32. 31. Rotting Oranges (Medium - Multi-Source BFS) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Multi-Source BFS: Enqueue all rotten oranges 2 initially and count fresh oranges 1. Process queue level by level (1 minute step). Rot adjacent fresh oranges and enqueue them.*

```swift
class RottingOranges {
    static func orangesRotting(_ grid: [[Int]]) -> Int {
        var g = grid, R = g.count, C = g[0].count, q = [(Int, Int)](), fresh = 0
        for r in 0..<R {
            for c in 0..<C {
                if g[r][c] == 2 { q.append((r, c)) }
                else if g[r][c] == 1 { fresh += 1 }
            }
        }
        if fresh == 0 { return 0 }
        var mins = 0, dirs = [(1,0),(-1,0),(0,1),(0,-1)]
        while !q.isEmpty && fresh > 0 {
            mins += 1
            for _ in 0..<q.count {
                let (r, c) = q.removeFirst()
                for (dr, dc) in dirs {
                    let nr = r + dr, nc = c + dc
                    if nr>=0 && nc>=0 && nr<R && nc<C && g[nr][nc] == 1 {
                        g[nr][nc] = 2; fresh -= 1; q.append((nr, nc))
                    }
                }
            }
        }
        return fresh == 0 ? mins : -1
    }
}
```

Time Complexity: O(M * N) - Every cell visited in BFS

Space Complexity: O(M * N) - Queue space for grid cells

#33. Pacific Atlantic Water Flow (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Reverse ocean border DFS: Start DFS from Pacific borders (top/left) and Atlantic borders (bottom/right). Water flows uphill (nextHeight >= currHeight). Result cells are reachable by both oceans.*

```
class PacificAtlantic {
    static func solve(_ heights: [[Int]]) -> [[Int]] {
        guard !heights.isEmpty else { return [] }
        let R = heights.count, C = heights[0].count
        var pac = Array(repeating: Array(repeating: false, count: C), count: R)
        var atl = Array(repeating: Array(repeating: false, count: C), count: R)
        func dfs(_ r: Int, _ c: Int, _ vis: inout [[Bool]], _ prev: Int) {
            if r<0 || c<0 || r>=R || c>=C || vis[r][c] || heights[r][c] < prev { return }
            vis[r][c] = true
            dfs(r+1,c,&vis,heights[r][c]); dfs(r-1,c,&vis,heights[r][c])
            dfs(r,c+1,&vis,heights[r][c]); dfs(r,c-1,&vis,heights[r][c])
        }
        for c in 0..<C { dfs(0, c, &pac, heights[0][c]); dfs(R-1, c, &atl, heights[R-1][c]) }
        for r in 0..<R { dfs(r, 0, &pac, heights[r][0]); dfs(r, C-1, &atl, heights[r][C-1]) }
        var res = [[Int]]()
        for r in 0..<R { for c in 0..<C { if pac[r][c] && atl[r][c] { res.append([r, c]) } } }
        return res
    }
}
```

Time Complexity: O(M * N) - DFS from ocean borders **Space Complexity:** O(M * N) - Matrix for ocean visited flags

#34. Graph Valid Tree (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *A graph of n nodes is a valid tree iff: (1) edges.count == n - 1, and (2) it is fully connected with no cycles (DFS starting from node 0 visits all n nodes without visiting parent).*

```
class GraphValidTree {
    static func validTree(_ n: Int, _ edges: [[Int]]) -> Bool {
        if edges.count != n - 1 { return false }
        var adj = [Int: [Int]]()
        for e in edges {
            adj[e[0], default: []].append(e[1]); adj[e[1], default: []].append(e[0])
        }
        var visited = Set<Int>()
        func dfs(_ curr: Int, _ parent: Int) -> Bool {
            if visited.contains(curr) { return false }
            visited.insert(curr)
            for neighbor in adj[curr] ?? [] {
                if neighbor == parent { continue }
                if !dfs(neighbor, curr) { return false }
            }
            return true
        }
        return dfs(0, -1) && visited.count == n
    }
}
```

Time Complexity: O(V + E) - DFS graph traversal **Space Complexity:** O(V + E) - Adjacency list + visited set

• Day 6: Dynamic Programming & Heap

#35. House Robber (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *1D DP state: At house i, max stolen money is max(prev1, prev2 + num). Maintain prev1 (max up to i-1) and prev2 (max up to i-2) across iteration.*

```
class HouseRobber {
    static func rob(_ nums: [Int]) -> Int {
        var prev1 = 0, prev2 = 0
        for num in nums {
            let current = max(prev1, prev2 + num)
            prev2 = prev1; prev1 = current
        }
        return prev1
    }
}
```

Time Complexity: O(N) - Single pass over houses **Space Complexity:** O(1) - Rolling variables

#36. Coin Change (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Unbounded knapsack bottom-up DP table dp of size amount + 1 initialized to amount + 1. Base case dp[0] = 0. For i in 1...amount, dp[i] = min(dp[i], dp[i - coin] + 1).*

```
class CoinChange {
    static func solve(_ coins: [Int], _ amount: Int) -> Int {
        guard amount > 0 else { return 0 }
        var dp = Array(repeating: amount + 1, count: amount + 1)
        dp[0] = 0
        for i in 1...amount {
            for coin in coins {
                if i - coin >= 0 { dp[i] = min(dp[i], dp[i - coin] + 1) }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount]
    }
}
```

Time Complexity: O(Amount * Coins.count) - Inner loop over coin types **Space Complexity:** O(Amount) - 1D DP table array

#37. 37. Longest Increasing Subsequence (Medium - Patience Sorting) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Maintain tails array storing smallest tail value of all increasing subsequences of length i+1. Binary search position of num in tails: replace element or append.*

```swift
class LongestIncreasingSubsequence {
    static func lengthOfLIS(_ nums: [Int]) -> Int {
        var tails = [Int]()
        for num in nums {
            var left = 0, right = tails.count
            while left < right {
                let mid = left + (right - left) / 2
                if tails[mid] < num { left = mid + 1 }
                else { right = mid }
            }
            if left == tails.count { tails.append(num) }
            else { tails[left] = num }
        }
        return tails.count
    }
}
```

Time Complexity: O(N log N) - Binary search inside linear loop

Space Complexity: O(N) - Space for tails array

#38. Word Break (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Boolean DP table dp[i] representing if string prefix s[0..<i] can be segmented. Base case dp[0] = true. For j from 0 to i-1, if dp[j] && wordSet.contains(s[j..<i]), set dp[i] = true.*

```swift
class WordBreak {
    static func solve(_ s: String, _ wordDict: [String]) -> Bool {
        let dict = Set(wordDict), n = s.count, chars = Array(s)
        var dp = Array(repeating: false, count: n + 1)
        dp[0] = true
        for i in 1...n {
            for j in 0..<i {
                if dp[j] && dict.contains(String(chars[j..<i])) {
                    dp[i] = true; break
                }
            }
        }
        return dp[n]
    }
}
```

Time Complexity: O(N^2 * L) - Substring slice and set check

Space Complexity: O(N + D) - DP array and dictionary hash set

#39. 39. Kth Largest Element in an Array (Medium - QuickSelect) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *QuickSelect algorithm: Partition array around pivot element. If pivot index equals target rank (count - k), return pivot value. Otherwise recurse into left or right partition.*

```
class KthLargestElement {
    static func findKthLargest(_ nums: [Int], _ k: Int) -> Int {
        var copy = nums, target = copy.count - k
        func quickSelect(_ l: Int, _ r: Int) -> Int {
            if l == r { return copy[l] }
            let p = partition(&copy, l, r)
            if p == target { return copy[p] }
            else if p < target { return quickSelect(p + 1, r) }
            else { return quickSelect(l, p - 1) }
        }
        return quickSelect(0, copy.count - 1)
    }
    private static func partition(_ nums: inout [Int], _ l: Int, _ r: Int) -> Int {
        let pivot = nums[r]; var i = l
        for j in l..<r {
            if nums[j] <= pivot { nums.swapAt(i, j); i += 1 }
        }
        nums.swapAt(i, r)
        return i
    }
}
```

Time Complexity: O(N) Average, O(N^2) Worst Case - QuickSelect expected time

Space Complexity: O(1) - In-place partition

#40. 40. Top K Frequent Elements (Medium - Bucket Sort) (Medium) [■ Open LeetCode Problem]

Logic & Intuition: *Count frequency of each number into hash map. Create bucket array buckets where index represents frequency count. Iterate buckets backwards from highest frequency to collect top k elements.*

```
class TopKFrequent {
    static func solve(_ nums: [Int], _ k: Int) -> [Int] {
        var counts = [Int: Int]()
        for num in nums { counts[num, default: 0] += 1 }
        var buckets = Array(repeating: [Int](), count: nums.count + 1)
        for (num, freq) in counts { buckets[freq].append(num) }
        var res = [Int]()
        for i in (0...nums.count).reversed() {
            for num in buckets[i] {
                res.append(num); if res.count == k { return res }
            }
        }
        return res
    }
}
```

Time Complexity: O(N) - Bucket sort linear sweep

Space Complexity: O(N) - Hash map and bucket storage

Day 7: Mock Interview & Final Revision Guide

Day 7 is dedicated to consolidating all learned patterns and performing timed mock interview simulations. Do not learn new topics on Day 7. Focus on clean communication, Big-O mastery, and edge-case validation.

■ 7-Step Interview Response Framework

1. **Clarify Requirements:** Confirm array length, range of integers, duplicates, nulls, and expected return types.
2. **State Naive / Brute Force Solution:** State the naive approach and analyze its Time & Space complexity out loud.
3. **Identify Optimal Pattern:** Transition to the optimal pattern (e.g. Hash Map, Two Pointers, BFS, DP).
4. **Discuss Time vs Space Trade-offs:** Explain why spending memory improves time (e.g. O(N) space map reduces time to O(N)).
5. **Write Clean Swift Code:** Write idiomatic Swift with type inference, clean optionals, and guard statements.
6. **Dry Run with Sample Inputs:** Walk through your code line-by-line using a short test array.
7. **Test Edge Cases:** Test empty input, single element, negative numbers, duplicates, and max boundaries.

■ Master Data Structure Complexity Reference Table

Data Structure / Algorithm	Access	Search	Insertion / Deletion	Space Complexity
Array / Contiguous Buffer	O(1)	O(N)	O(N)	O(N)
Hash Map / Set	N/A	O(1) Avg	O(1) Avg	O(N)
Singly Linked List	O(N)	O(N)	O(1) at Head	O(N)
Binary Search Tree (BST)	O(H)	O(H)	O(H)	O(N)
Min / Max Heap	O(1) Top	O(N)	O(log N) Push/Pop	O(N)
Graph BFS / DFS	N/A	O(V + E)	O(V + E)	O(V + E)
Binary Search	N/A	O(log N)	N/A	O(1) Iterative

■ Final Checklist Before Interview

✔■ Ensure 100% test assertions pass in WBD_7Day_DSAPrep.playground.

✔■ Memorize binary search templates (left <= right vs left < right).

✔■ Practice speaking out loud while coding in Swift.

✔■ Stay calm, communicate clearly, and enjoy the problem-solving process!